IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Regularizar: frenar al modelo

Ridge y Lasso cuando hay muchas variables y pocos datos

Sprint 7 · S07_02 · 35 min en clase (ejercicios 1, 2 y 3)

EL PROBLEMA · SECCIÓN 1

Muchas columnas, pocas filas

120

reservas para entrenar: el alojamiento es nuevo

35

columnas, tras pasar las categorías a 0/1

665

columnas al añadir todas las combinaciones de grado 2

ASÍ CRECEN LAS COLUMNAS

```
PolynomialFeatures(2, include_bias=False)
[2, 3]  →  [2, 3, 4, 6, 9]
# a, b, a², a·b, b²: de 2 columnas a 5
```

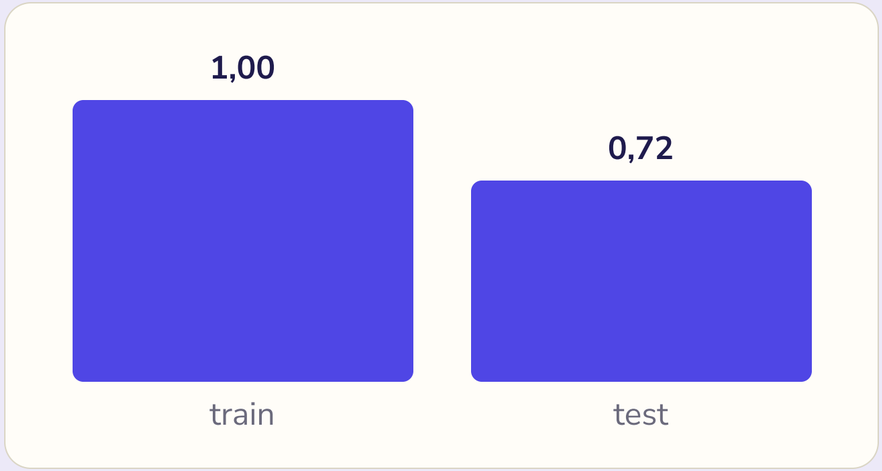
Más columnas que filas: el modelo puede memorizar el ruido, como quien se aprende el examen de memoria.

SOBREAJUSTE · EJERCICIO 1

Train perfecto, test mediocre

x	precio real	recta	polinomio grado 4
1	62	62.6	62.0
2	70	67.3	70.0
3	68	72.0	68.0
4	79	76.7	79.0
5	81	81.4	81.0
6 (nueva)	—	86.1	7.0

ols del cuaderno (R^2 del modelo sin regularizar):



5 puntos, 5 coeficientes: el polinomio acierta todos… y para x = 6 predice 7. Brecha del cuaderno: 1,00 – 0,72 = 0,28.

LA IDEA · SECCIÓN 2

Castigar los coeficientes grandes

30 reservas simuladas con 4 variables **escaladas**: solo **suite** y **alta** influyen; las otras dos son ruido.

Coeficientes	suite	alta	ruido 1	ruido 2
Sin penalizar	5.23	3.11	0.23	0.67
Ridge, alpha 1	5.05	3.00	0.21	0.60
Ridge, alpha 10	3.87	2.29	0.08	0.22
Lasso, alpha 0.5	4.61	2.51	0	0
Lasso, alpha 2	3.14	1.03	0	0

Ridge (L2)

Castigo: **alpha** · suma de **cuadrados**. Encoge todos los coeficientes; ninguno llega a 0.

Lasso (L1)

Castigo: **alpha** · suma de **valores absolutos**. Lleva a 0 los que no sirven.

ELEGIR ALPHA · EJERCICIO 2

Que elija la validación cruzada, no el test

```
ridge = modelo_con(RidgeCV(alphas=np.logspace(-1, 4, 30))).fit(X_train, y_train)
alpha_ridge = ridge[-1].alpha_
r2_test_ridge = ridge.score(X_test, y_test)
```

np.logspace(-1, 4, 30) = 30 valores entre 0,1 y 10 000 que crecen por factores (como **np.logspace(0, 3, 4)**: 1, 10, 100, 1000).

25,9
alpha elegido por **RidgeCV**

0,728
R² en test con Ridge

0,716
R² en test sin regularizar

El test no se toca hasta el final: si eliges alpha mirando el test, el examen ya no vale.

LASSO · EJERCICIO 3

Lasso se queda con lo que importa

```
lasso = modelo_con(LassoCV(cv=5, max_iter=50000, random_state=SEMILLA)).fit(X_train, y_train)
r2_test_lasso = lasso.score(X_test, y_test)
n_no_cero = int(np.sum(lasso[-1].coef_ != 0))
```

Las 3 que más pesan (cuaderno)	coef.
cat__tipo_habitacion_suite	16.16
cat__tipo_habitacion_estandar	−14.03
num__temporada_alta	7.18

Resultado en test

R^2 con Lasso: **0,887**

Coeficientes distintos de 0: **27 de 665**

Las más importantes son el tipo de habitación y la temporada: justo lo que mueve el precio.

COMPARATIVA · SECCIÓN 5 (CASA)

Ninguno gana siempre

Modelo	R^2 train	R^2 test
Sin regularizar	1.000	0.716
Ridge	0.998	0.728
Lasso	0.957	0.887

Ridge

Mejor cuando **muchas** variables aportan un poco.

Lasso

Mejor cuando **pocas** lo explican casi todo (nuestro caso) y se explica fácil.

ElasticNet

Mezcla las dos penalizaciones: útil con variables muy **correlacionadas**.

Fíjate en la brecha: Lasso baja un poco en train (0,957) y gana mucho en test.

MINI-RETO (CASA)

¿Cuántos datos hacen falta?

```
tamanos = [60, 120, 300, 1000]
X_res, X_eval, y_res, y_eval = train_test_split(
    X, y, test_size=1000, random_state=SEMILLA)
for n in tamanos:
    # ajusta OLS y Lasso con X_res.iloc[:n], y_res.iloc[:n]
    # y puntúa SIEMPRE con X_eval, y_eval
```

Qué entregas

La tabla **curva**: una fila por tamaño y las columnas **ols** y **lasso**.

Mismo examen para todos

Las mismas **1000 reservas** de evaluación: así la comparación es justa.

Cuidado con n = 300

Con 665 columnas, mínimos cuadrados se vuelve **inestable**: un R^2 muy negativo no es un error.

HOY HAS APRENDIDO

Frenar al modelo para que generalice

Reconocer

Muchas variables y pocos datos: train perfecto, test mediocre

Regularizar

Ridge encoge los coeficientes; Lasso además los lleva a 0

Elegir

alpha con **RidgeCV**/**LassoCV**, y siempre con las variables escaladas

Siguiente: S07_03 · regresión logística y cancelaciones